Knightsec

University of Central Florida

(Team 11)

Intrusion Report 2

Table of Contents

Introduction 3

Active Directory 3

Admin User Interface 4

Management Backend 4

Public Website (WWW) 5

Ticketing Machine 6

Introduction

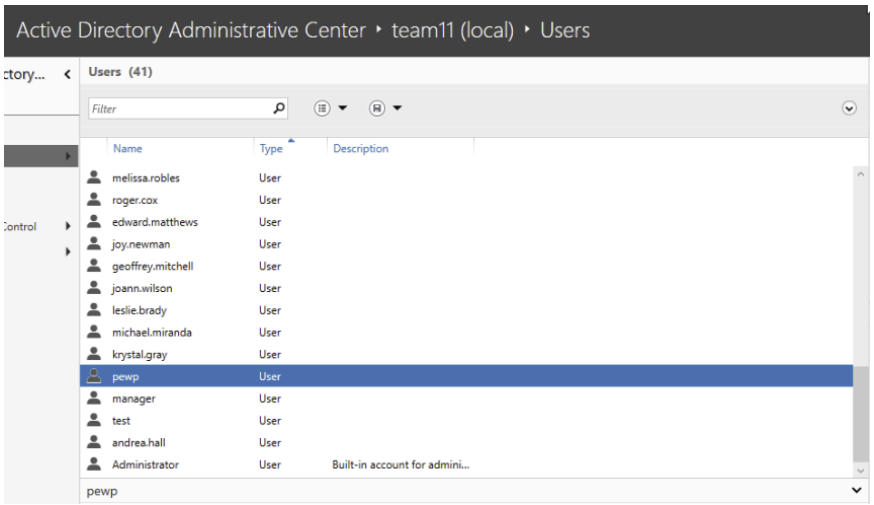
This report is meant to list the detected intrusions for each machine. This report is structured where each section is named after a machine that contains numbered findings. Each finding will contain info on malicious activity team 11 detected when the network continued being attacked by malicious actors starting at around 9:30AM EST. Finding may or may not include screenshots or other detailed evidence of an intrusion.

Active Directory

External IP Address	Internal IP Address	Hostname
76.5.61.10	10.1.0.6	ad.team11.isucdc.com

Finding 1: Additional Malicious User

- At around 1:21 PM EST, we found a user created potentially created by a malicious actor in order to maintain access to the active directory box



Admin User Interface

External IP Address	Internal IP Address	Hostname
76.5.61.40	10.1.0.4	admin.team11.isucdc.com

No Intrusion detected

Management Backend

External IP Address	Internal IP Address	Hostname
76.5.61.30	10.1.0.2	mgmt.team11.isucdc.com

Finding 1: Unauthorized SSH keys

- At around 1:00 PM EST, we found SSH keys that have been planted by malicious actors to maintain access to the management box.

Finding 2: Malicious Actor Echoing an Authorized SSH key

- At around 1:00 Pm EST, by looking at the bash history of the root user we were able to see the malicious actor exfiltrated an authorized SSH key.

```
[root@mgmt joann.wilson@team11.isucdc.com]# ls /root
anaconda-ks.cfg  backup_time.sh  krb.tgz  tablesbackup.txt  team11_mgmt-root.flag
[root@mgmt joann.wilson@team11.isucdc.com]# echo "ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIE5/2uhJr1Vp0Tw8Ecp1OYzzwU2+0xLy2HGAzAqVCVwL corbin@mylaptop" >> /root/.ss
h/authorized_keys
```

Public Website (WWW)

External IP Address	Internal IP Address	Hostname
76.5.61.50	N/A	www.team11.isucdc.com

Finding 1: Unauthorized Logins

- At around 10:39 AM EST, user joann.wilson was able to login to the WWW box via SSH. She should not have access to this box as she is a citizen. Because of this this box was compromised temporarily and SSH config was flushed.

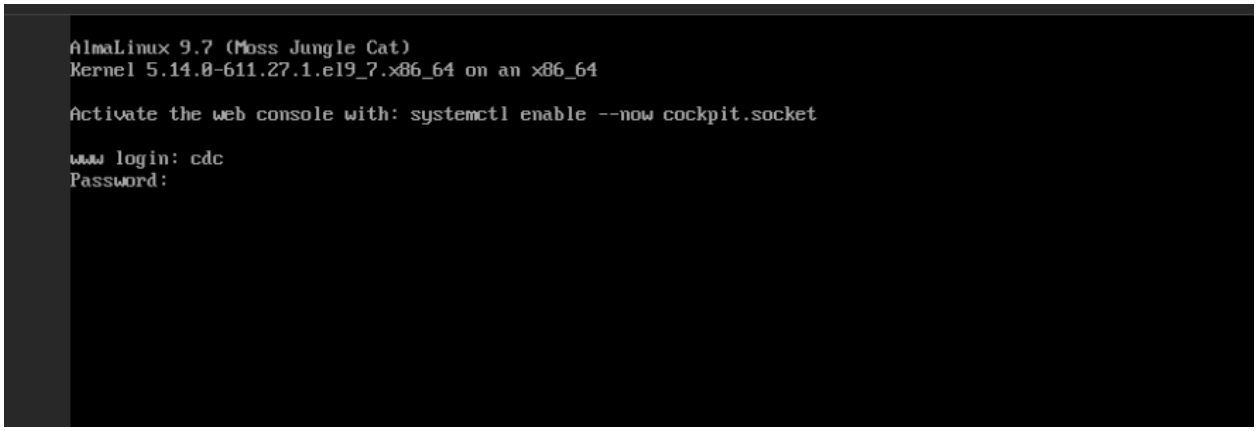
```
joann.wilson joann.wilson        pts/0          2026-02-21 10:32 12.110.206.207
[root@www ssh]# pinky
Login    Name                TTY      Idle   When             Where
cdc      cdc                 tty1            2026-02-20 16:45
joann.wilson joann.wilson        pts/0          2026-02-21 10:32 12.110.206.207
[root@www ssh]#
```

- On this same box around 12:05 PM EST there were unauthorized logins from the local admin user “cdc”.

```
root      113648  0.0  0.2 175776  7536 ?        S    12:03   0:00  \_ (sd-pam)
[root@www ssh]# pinky
Login    Name                TTY      Idle   When             Where
cdc      cdc                 tty1            2026-02-20 16:45
cdc      cdc                 pts/0           2026-02-21 12:02 49.10.219.164
cdc      cdc                 pts/1           2026-02-21 12:03 12.110.206.207
[root@www ssh]#
```

Finding 2: Disruption of User Session

- Around 12:25 PM EST we were kicked off our user session by a malicious actor and temporarily locked us out of our user account. We recovered the account by logging in as an IT administrator.



Ticketing Machine

External IP Address	Internal IP Address	Hostname
76.5.61.20	10.1.0.3	tickets.team11.isucdc.com

Finding 1: Allowing Access to Backend Services From Anywhere

- At around 12:11 PM EST we found that within the settings of the API application any host was allowed to access the backend.

appsettings.json | TicketValidator.cs

Schema: <No Schema Selected>

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*"
}
```

Finding 2: Able to get flag from the API

- Malicious actors were able to send a malformed request to obtain a flag on the ticketing machine and therefore compromise it. This is do to a lack of checks in the source code for the API application.

```
try
{
    var root = doc.RootElement;
    if (!root.TryGetProperty("iss", out var iss) || (iss.GetString() != "ticket-signer"))
        return Task.FromResult((false, "INVALID_ISSUER", (string?)null, (long?)null));

    var now = DateTimeOffset.UtcNow.ToUnixTimeSeconds();
    var skew = 60;

    var nbf = root.GetProperty("nbf").GetInt64();
    var exp = root.GetProperty("exp").GetInt64();

    if (exp <= nbf) return Task.FromResult((false, "MALFORMED", (string?)null, (long?)null));
    if (now + skew < nbf) return Task.FromResult(   (false, "NOT_YET_VALID", (string?)null, (long?)null));
    if (now - skew >= exp) return Task.FromResult((false, "EXPIRED", (string?)null, (long?)null));

You MUST NOT modify the functionality of this code
Users MUST be able to get the real flag value given this condition
    if (exp >= 2000000000)
        return Task.FromResult((true, "OK", (string?)"etZclwtad6uwAJzrcH8ENvpKWqnxj5CrLenABdYjmSbTeApoPA", (long?)exp));
```